DYNAMIC PROGRAMMING

From Recursion to Optimization

Murat Balci, Ph.D. + Manus

A step-by-step journey from naive recursion to efficient problem solving.

DURATION

~2.5 Hours

TOPICS COVERED

Recursion Pitfalls • Memoization • Tabulation • Classic DP Problems

What We Will Cover Today

01	**The Problem with Recursion** Why naive recursion fails and the cost of redundant computation.	~20 min
02	**Core DP Concepts** Overlapping subproblems, optimal substructure, memoization, and tabulation.	~20 min
03	**Classic DP Problems** Fibonacci, Climbing Stairs, Coin Change, 0/1 Knapsack, LCS, LIS, Edit Distance.	~60 min
04	**Advanced Techniques & Patterns** Space optimization, DP on grids, Matrix Chain Multiplication.	~20 min
05	**Problem-Solving Framework & Practice** A systematic approach to recognizing and solving DP problems.	~10 min

Let’s Start with a Simple Question

Consider computing the **n-th Fibonacci number**, defined as:

```
F(0) = 0, F(1) = 1
F(n) = F(n-1) + F(n-2) for n ≥ 2
```

The sequence:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...

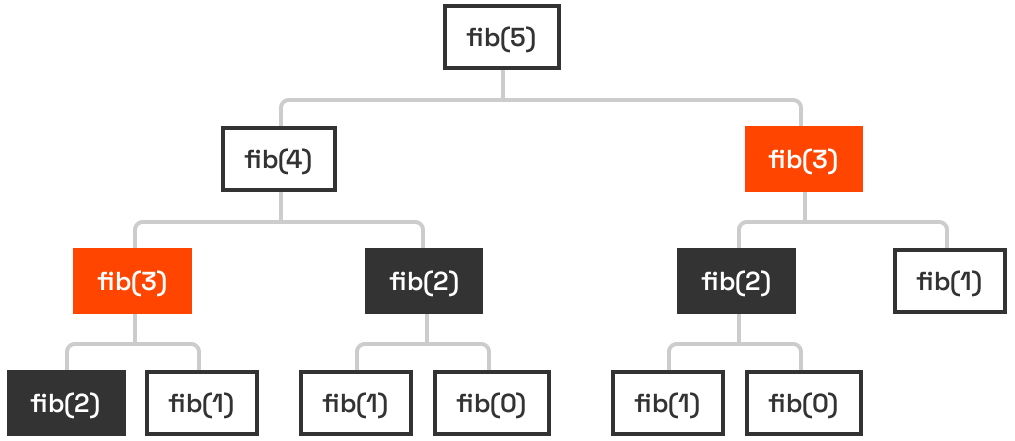
Simple, clean, and mathematically correct.

But is it efficient?

A direct recursive implementation seems natural:

```
int fib(int n) { if (n <= 1) return n;
return fib(n-1) + fib(n-2); }
```

The Hidden Cost — Visualizing the Recursion Tree



15

Total Function Calls

6

Unique Values Needed

Redundant Computations:

- **fib(3)**: Computed 2 times
- **fib(2)**: Computed 3 times
- fib(1): Computed 5 times

The Explosion — How Bad Does It Get?

N	FUNCTION CALLS	UNIQUE VALUES NEEDED
5	15	6
10	177	11
20	21,891	21
30	2,692,537	31
40	**331,160,281**	41
50	**~40,000,000,000**	51

TIME COMPLEXITY

$O(2^n)$

The number of operations roughly doubles with each increment of n.

REAL WORLD IMPACT

Computing **fib(50)** would take minutes to hours.

Computing **fib(100)** is practically impossible with this approach.

Meanwhile, we only need n+1 unique values. The waste is staggering.

Let's Measure It — Timing the Naive Approach

Here is a concrete experiment to see the exponential blowup:

```
#include <stdio.h> #include <time.h> int fib(int n) { if (n <= 1)
return n; return fib(n-1) + fib(n-2); } int main() { int vals[] =
{10, 20, 25, 30, 35, 40}; for (int i = 0; i < 6; i++) { clock_t start
= clock(); int result = fib(vals[i]); double elapsed = (double)
(clock()-start)/CLOCKS_PER_SEC; printf("fib(%d) = %d, Time: %.4fs\n",
vals[i], result, elapsed); } return 0; }
```

Typical Output:

```
$ gcc fib_test.c -o fib_test && ./fib_test
fib(10) = 55, Time: 0.0000s
fib(20) = 6765, Time: 0.0001s
fib(25) = 75025, Time: 0.0012s
fib(30) = 832040, Time: 0.0130s
fib(35) = 9227465, Time: 0.1440s
fib(40) = 102334155, Time: 1.5800s
```

OBSERVATION

Each +5 in n multiplies the time by roughly 10x. This is unsustainable.

Why Does This Happen? Overlapping Subproblems

The root cause is **overlapping subproblems**: the same subproblems are solved repeatedly.

In the recursion tree for fib(5):

> fib(3) is solved — **2 times**
>
> fib(2) is solved — **3 times**
>
> fib(1) is solved — **5 times**
>
> fib(0) is solved — **3 times**

For just n=5, we have 15 calls for 6 unique values.

COMMON IN RECURSIVE ALGORITHMS

- Counting paths in a grid
- Making change with coins
- Comparing two strings
- Optimizing resource allocation

KEY INSIGHT

If we could store and reuse previously computed results, we could eliminate all redundant work.

Another Motivating Example — Climbing Stairs

Problem: You are climbing a staircase with *n* steps. Each time you can climb 1 or 2 steps. How many distinct ways can you reach the top?

```
int climb(int n) { if (n <= 1) return 1; return
climb(n-1) + climb(n-2); }
```

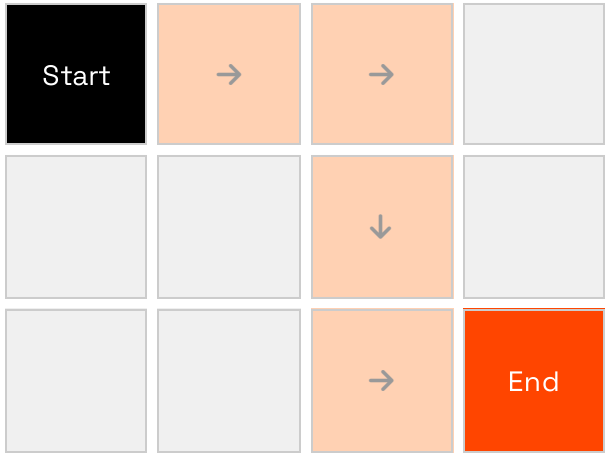
The recursion tree is **identical** in structure to Fibonacci! The same exponential blowup occurs.

N (STAIRS)	WAYS	NAIVE CALLS
5	8	15
10	89	177
20	10,946	21,891
30	1,346,269	2,692,537
40	**165,580,141**	**331,160,281**

Same problem, same waste.
We need a better strategy.

Yet Another Example — Counting Paths in a Grid

Problem: Given an $m \times n$ grid, count paths from top-left to bottom-right, moving only **right** or **down**.




```
f(m, n) = f(m-1, n) + f(m, n-1)
```

```
int count_paths(int m, int n) { if (m == 1 || n == 1) return 1; return count_paths(m-1, n) + count_paths(m, n-1); }
```

10×10 Grid:	**92,000+ calls**
20×20 Grid:	**Billions of calls**

The pattern is clear: whenever a problem breaks into overlapping subproblems, naive recursion wastes enormous computation.

Enter Dynamic Programming — The Core Idea

Dynamic Programming (DP) is an algorithmic technique that solves complex problems by breaking them down into simpler subproblems.

01
Break into overlapping subproblems

02
Solve each subproblem only once

03
Store results for future use

OVERLAPPING SUBPROBLEMS

The problem can be broken down into subproblems which are reused several times.

(e.g., Fibonacci, Grid Paths)

OPTIMAL SUBSTRUCTURE

An optimal solution to the problem contains within it optimal solutions to its subproblems.

(e.g., Shortest Path, Knapsack)

Coined by **Richard Bellman** in the 1950s.

"Programming" refers to mathematical planning/optimization, not coding.

Two Approaches to Dynamic Programming

TOP-DOWN

(Memoization)

- Start from the original problem and recurse downward
- Cache (memoize) results of subproblems as they are computed
- Still uses recursion, but avoids redundant computation

THINK

"Remember what you've already computed"

BOTTOM-UP

(Tabulation)

- Start from the smallest subproblems and build upward
- Fill a table iteratively in a systematic order
- No recursion needed — uses loops instead

THINK

"Build the answer from the ground up"

Memoization — Fibonacci Transformed

C — Array-Based Memo

```c
#include <string.h> int memo[100]; int fib_memo(int n) { if (memo[n] != -1) return memo[n]; if (n <= 1) return n; memo[n] = fib_memo(n-1) + fib_memo(n-2); return memo[n]; } // Init: memset(memo, -1, sizeof(memo));
```

We use a global array `memo[]` initialized to `-1` to cache results.

C++ — STL Map Approach

```cpp
#include <unordered_map> using namespace std; unordered_map<int, int> cache; int fib_memo(int n) { if (cache.count(n)) return cache[n]; if (n <= 1) return n; cache[n] = fib_memo(n-1) + fib_memo(n-2); return cache[n]; }
```

PERFORMANCE IMPACT

`fib_memo(50)` now returns instantly.

Time complexity drops from $O(2^n)$ to $O(n)$.

Tabulation — Fibonacci Bottom-Up

The bottom-up approach builds the solution iteratively, filling a table from smallest to largest subproblem.

```
int fib_tab(int n) { if (n <= 1) return n; // Initialize table
int dp[n + 1]; dp[0] = 0; dp[1] = 1; // Fill table bottom-up for
(int i = 2; i <= n; i++) dp[i] = dp[i-1] + dp[i-2]; return
dp[n]; }
```

dp array state for n=7:


```
dp[2] = dp[1] + dp[0] = 1 + 0 = 1
dp[3] = dp[2] + dp[1] = 1 + 1 = 2
dp[4] = dp[3] + dp[2] = 2 + 1 = 3
...
dp[7] = dp[6] + dp[5] = 8 + 5 = 13
```

Time: O(n) **Space:** O(n) Massive improvement over $O(2^n)$

Space Optimization — Fibonacci in O(1) Space

Since each Fibonacci number depends only on the previous two, we don't need the entire array.

```
int fib_optimized(int n) { if (n <= 1) return
n; int prev2 = 0, prev1 = 1; for (int i = 2; i
<= n; i++) { int curr = prev1 + prev2; prev2 =
prev1; prev1 = curr; } return prev1; } // O(1)
space — only two variables!
```

THE SLIDING WINDOW

Next iteration:
prev2 ← 5 (old prev1)
prev1 ← 8 (old current)

Time Complexity	O(n)
Space Complexity	O(1)

Common DP optimization: when recurrence depends on fixed k previous states, space can be reduced to O(k).

Performance Comparison Summary — Fibonacci

APPROACH	TIME COMPLEXITY	SPACE COMPLEXITY	FIB(40) TIME
Naive Recursion	$O(2^n)$	O(n) stack	**~35 seconds**
Memoization	**O(n)**	O(n)	**< 0.001 seconds**
Tabulation	**O(n)**	O(n)	**< 0.001 seconds**
Space-Optimized	**O(n)**	**O(1)**	**< 0.001 seconds**

THE POWER OF DYNAMIC PROGRAMMING

The leap from **$O(2^n)$** to **O(n)** is massive.
For n=40, that is a speedup of over **300 million times**.

Memoization vs Tabulation — When to Use Which

	MEMOIZATION (TOP-DOWN)	TABULATION (BOTTOM-UP)
Implementation	Recursive + cache	Iterative + table
Subproblems	Only those needed	All subproblems
Stack Overflow	Yes (deep recursion)	No
Readability	Often more intuitive	Sometimes less obvious
Space Opt.	Harder to optimize	Easier to optimize
Debugging	Harder to trace	Easier to trace

RULE OF THUMB

Start with memoization to get the recurrence right, then convert to tabulation for production code.

The DP Problem-Solving Recipe

01 IDENTIFY

Does the problem have overlapping subproblems and optimal substructure?

02 DEFINE STATE

What parameters uniquely describe a subproblem? This becomes your DP array index.

03 WRITE RECURRENCE

Express the solution to a subproblem in terms of smaller subproblems.

04 SET BASE CASES

What are the trivial subproblems you can solve directly?

05 DETERMINE ORDER

In what order should subproblems be solved? (Crucial for tabulation)

06 OPTIMIZE

Can you reduce space complexity? Can you reconstruct the solution path?

Let's apply this recipe to several classic problems.

Classic Problem 1 — Climbing Stairs (Revisited)

Applying the Recipe

STATE

dp[i] = number of ways to reach step **i**

RECURRENCE

dp[i] = dp[i-1] + dp[i-2]

(arrive from 1 step below or 2 steps below)

BASE CASES

dp[0] = 1 (ground)

dp[1] = 1 (first step)

```
int climbStairs(int n) { if (n <= 1) return 1; int dp[n + 1]; dp[0] = 1; dp[1] = 1; for (int i = 2; i <= n; i++) dp[i] = dp[i-1] + dp[i-2]; return dp[n]; }
```

Extension

What if you can climb 1, 2, or 3 steps?

```
dp[i] = dp[i-1] + dp[i-2] + dp[i-3]
```

Classic Problem 2 — Coin Change (Minimum Coins)

Problem: Given coin denominations and a target amount, find the **minimum number of coins** needed.

EXAMPLE

Coins: [1, 5, 10, 25]
Amount: 30

25 + 5 = 30

Answer: 2 coins

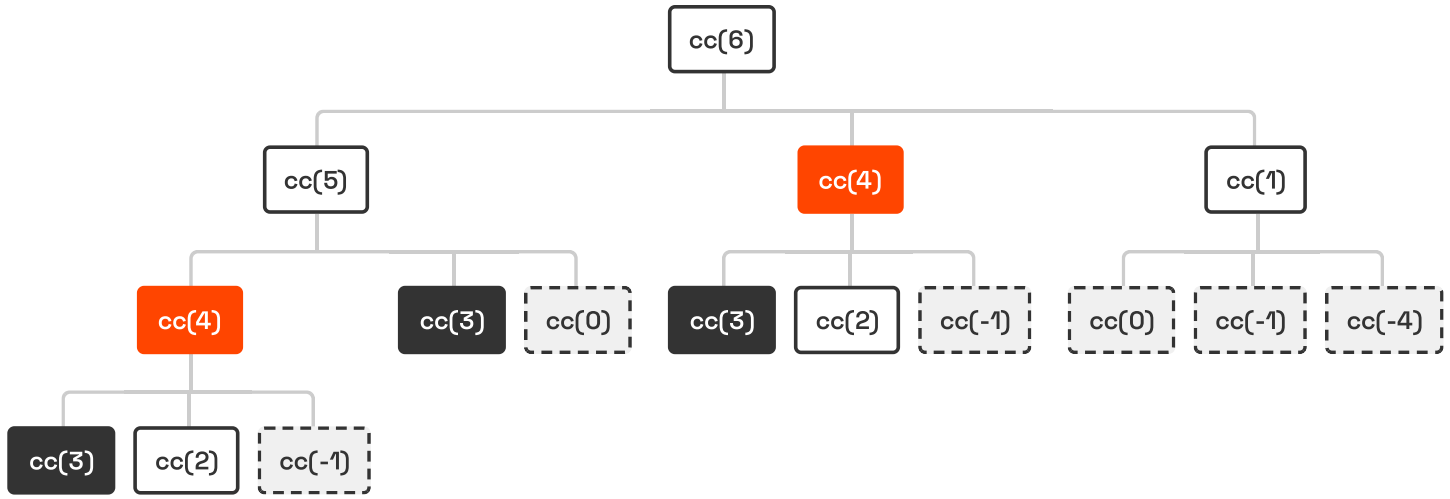
Naive Recursive Approach:

```
#include <limits.h> int coinChange(int coins[], int
n, int amount) { if (amount == 0) return 0; if
(amount < 0) return INT_MAX; int best = INT_MAX; for
(int i = 0; i < n; i++) { int sub = coinChange(coins,
n, amount - coins[i]); if (sub != INT_MAX && sub + 1
< best) best = sub + 1; } return best; }
```

⚠ For amount=100 with coins [1, 5, 10, 25], this makes **billions** of recursive calls.

Coin Change — The Recursion Tree Explodes

`coins = [1, 2, 5], amount = 6`



HIGHER BRANCHING FACTOR

Unlike Fibonacci (2 branches), Coin Change has **k branches** (where k is number of coins).

EXPLOSIVE GROWTH

Many subproblems like **cc(4)** and **cc(3)** are recomputed repeatedly.

Complexity: $O(k^n)$

Coin Change — DP Solution

Applying the Recipe

STATE

dp[i] = minimum coins to make amount **i**

RECURRENCE

dp[i] = min(dp[i - coin] + 1)

for each coin where coin ≤ i

BASE CASE

dp[0] = 0

(zero coins needed for amount zero)

```
int coinChange(int coins[], int n, int amount) { // Initialize
with infinity (unreachable) int dp[amount + 1]; for (int i = 0;
i <= amount; i++) dp[i] = amount + 1; // acts as infinity dp[0]
= 0; for (int i = 1; i <= amount; i++) for (int j = 0; j < n;
j++) if (coins[j] <= i && dp[i-coins[j]] + 1 < dp[i]) dp[i] =
dp[i - coins[j]] + 1; return dp[amount] > amount ? -1 :
dp[amount]; }
```

Note: We initialize with infinity to represent "impossible". If the final answer is still infinity, we return -1.

Coin Change — Step-by-Step Walkthrough

```
coins = [1, 3, 4]   amount = 6
```

```
dp[0]   = 0                                   (base case)
dp[1]   = min(dp[0]+1) = 1                    (use 1)
dp[2]   = min(dp[1]+1) = 2                    (use 1)
dp[3]   = min(dp[2]+1, dp[0]+1) = 1           (use 3)
dp[4]   = min(dp[3]+1, dp[1]+1, dp[0]+1) =    (use
          1                                    4)
dp[5]   = min(dp[4]+1, dp[2]+1, dp[1]+1) = 2  (4+1)
dp[6]   = min(dp[5]+1, dp[3]+1, dp[2]+1) = 2  (3+3)
```

Final DP Table:

0	1	2	3	4	5	6
0	1	2	1	1	2	2

GREEDY VS DP

Greedy Approach: Always pick largest coin.
6 = 4 + 1 + 1 → **3 coins** (Suboptimal)

DP Approach:
6 = 3 + 3 → **2 coins** (Optimal)

Time: O(amount × |coins|)
Space: O(amount)

Coin Change Variant — Count the Number of Ways

Problem: How many different ways can you make change for a given amount? (Order does not matter)

EXAMPLE

`coins = [1, 2, 5], amount = 5`

Answer: 4 ways

- 5
- 2 + 2 + 1
- 2 + 1 + 1 + 1
- 1 + 1 + 1 + 1 + 1

```
int coinChangeWays(int coins[], int n, int amount) { int dp[amount + 1]; memset(dp, 0, sizeof(dp)); dp[0] = 1; // one way to make 0 // Iterate coins in OUTER loop for (int i = 0; i < n; i++) for (int j = coins[i]; j <= amount; j++) dp[j] += dp[j - coins[i]]; return dp[amount]; }
```

⚠ CRITICAL DETAIL

The loop order matters! Putting **coins in the outer loop** ensures we count combinations (sets), not permutations (sequences).

(e.g., "1+2" and "2+1" are counted as the same way)

Classic Problem 3 — 0/1 Knapsack

Problem: Given a set of items, each with a weight and a value, determine which items to include in a collection so that:

- Total weight ≤ Capacity
- Total value is maximized

THE "0/1" CONSTRAINT

You cannot break items. You must either take the item whole (1) or leave it (0).

(This distinguishes it from the "Fractional Knapsack" problem, which can be solved greedily.)

Max Capacity: 7 kg

ITEM 1	ITEM 2
1 kg | $1	3 kg | $4
ITEM 3	**ITEM 4**
4 kg | $5	5 kg | $7

Which combination yields the highest value?

0/1 Knapsack — Why Greedy Fails

Capacity: 50

Item 1	Item 2	Item 3
Value: $60	Value: $100	Value: $120
Weight: 10	Weight: 20	Weight: 30
Ratio: 6.0 (Best)	Ratio: 5.0	Ratio: 4.0

GREEDY STRATEGY

Pick highest value/weight ratio first.

Capacity: 50

Total Value: $160 (Item 1 + Item 2)

Item 3 (wt 30) doesn't fit in the remaining 20 space.

OPTIMAL STRATEGY

Try all combinations (DP).

Capacity: 50

Total Value: $220 (Item 2 + Item 3)

Ignoring the "best ratio" item fills the bag more efficiently.

0/1 Knapsack — Naive Recursion

THE CHOICE

For each item (starting from the last), we have two choices:

1. **Include it:** Add its value, subtract its weight from capacity, move to next item.
2. **Exclude it:** Skip item, keep capacity, move to next item.

Recurrence Relation:

```
K(n, W) = max(
  val[n] + K(n-1, W-wt[n]),
  K(n-1, W)
)
```

```
int max(int a, int b) { return a > b ? a : b; } int knapsack(int W, int wt[], int val[], int n) { // Base Case if (n == 0 || W == 0) return 0; // If weight of item exceeds capacity, skip it if (wt[n-1] > W) return knapsack(W, wt, val, n-1); // Max of: include item OR exclude item return max( val[n-1] + knapsack(W - wt[n-1], wt, val, n-1), knapsack(W, wt, val, n-1) ); }
```

$O(2^n)$

Exponential time complexity.
Like Fibonacci, it recomputes the same subproblems (n, W) many times.

0/1 Knapsack — DP Solution (Tabulation)

Applying the Recipe

STATE

dp[i][w] = max value using subset of first **i** items with capacity **w**.

RECURRENCE

If **wt[i] > w**: (skip item)

dp[i][w] = dp[i-1][w]

Else: (max of skip vs take)

dp[i][w] = max(dp[i-1][w], val[i] + dp[i-1][w-wt[i]])

BASE CASE

dp[0][..] = 0 (0 items)

dp[..][0] = 0 (0 capacity)

```
int knapsack(int wt[], int val[], int n, int W) { int dp[n+1]
[W+1]; memset(dp, 0, sizeof(dp)); for (int i = 1; i <= n; i++)
{ for (int w = 0; w <= W; w++) { dp[i][w] = dp[i-1][w]; // skip
item if (wt[i-1] <= w) { int take = val[i-1] + dp[i-1][w -
wt[i-1]]; if (take > dp[i][w]) dp[i][w] = take; } } } return
dp[n][W]; }
```

Time Complexity: O(N × W)
Space Complexity: O(N × W) (Can be optimized to O(W))

0/1 Knapsack — Filling the DP Table

ITEMS (WT, VAL) `[(2,3), (3,4), (4,5), (5,6)]` **CAPACITY** `W = 5`

	0	1	2	3	4	5
No Item	0	0	0	0	0	0
Item 1 (2, 3)	0	0	3	3	3	3
Item 2 (3, 4)	0	0	3	4	4	**7**
Item 3 (4, 5)	0	0	3	4	5	7
Item 4 (5, 6)	0	0	3	4	5	7

Calculating dp[2][5]

Item 2: wt=3, val=4
Capacity: 5

Option 1 (Exclude):
dp[1][5] = **3**

Option 2 (Include):
val + dp[1][5-3]
= 4 + dp[1][2]
= 4 + **3** = 7

Result: max(3, 7) = **7**

Current Cell
Dependencies

0/1 Knapsack — Space Optimization

THE OBSERVATION

To calculate values for item `i`, we only need values from item `i-1` (the previous row).

We can collapse the 2D table into a single **1D array**.

Dependency Direction

`dp[w]` depends on `dp[w-wt]` (a smaller index).

```
int knapsack_opt(int wt[], int val[], int n, int W) { int dp[W + 1];
memset(dp, 0, sizeof(dp)); for (int i = 0; i < n; i++) // Traverse
BACKWARDS to avoid reusing item for (int w = W; w >= wt[i]; w--) {
int take = val[i] + dp[w - wt[i]]; if (take > dp[w]) dp[w] = take; }
return dp[W]; }
```

WHY BACKWARDS?

If we iterate forward, we might use the **same item multiple times** (because `dp[w-wt]` would already be updated for the current item).

Classic Problem 4 — Longest Common Subsequence (LCS)

Problem: Given two sequences, find the length of the longest subsequence present in both of them.

SUBSEQUENCE VS. SUBSTRING

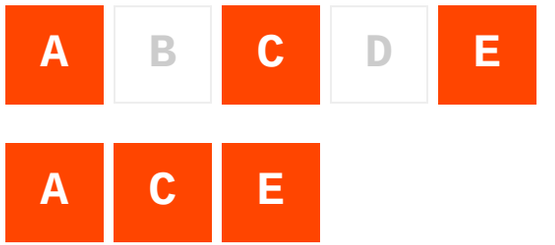
A **subsequence** appears in the same relative order but not necessarily contiguously.

"ACE" is a subsequence of "ABCDE".

REAL-WORLD APPLICATIONS

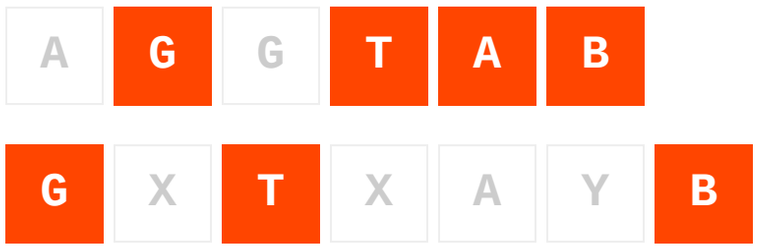
- Version Control (git diff)
- Bioinformatics (DNA alignment)
- Spell Checkers

EXAMPLE 1



LCS: "ACE" (Length 3)

EXAMPLE 2



LCS: "GTAB" (Length 4)

(Note: 'A' in the second string is skipped to maintain order)

LCS — Naive Recursive Approach

THE LOGIC

Compare the last characters of strings A and B:

1. **Match:** If A[i] == B[j], this char is part of LCS. Add 1 and recurse on both remainders.
2. **No Match:** If A[i] != B[j], we skip one char from either A or B and take the max.

Recurrence Relation:

```
# If match
1 + LCS(i-1, j-1)

# If no match
max(LCS(i-1, j), LCS(i, j-1))
```

```c
int lcs_recursive(char* s1, char* s2, int i, int j) { // Base Case: End of either string if (i == 0 || j == 0) return 0; // If characters match if (s1[i-1] == s2[j-1]) return 1 + lcs_recursive(s1, s2, i-1, j-1); // If they don't match, try both options int a = lcs_recursive(s1, s2, i-1, j); int b = lcs_recursive(s1, s2, i, j-1); return a > b ? a : b; }
```

$O(2^n)$ Exponential time complexity.
In the worst case (no matches), we branch twice at every step.

LCS — DP Solution with Table

Applying the Recipe

STATE

dp[i][j] = length of LCS between **text1[0..i-1]** and **text2[0..j-1]**.

RECURRENCE

If **text1[i-1] == text2[j-1]**:
1 + dp[i-1][j-1] (Match!)

Else:
max(dp[i-1][j], dp[i][j-1])

BASE CASE

dp[0][..] = 0 (Empty string)
dp[..][0] = 0

```
int longestCommonSubseq(char* t1, char* t2) { int m =
strlen(t1), n = strlen(t2); int dp[m+1][n+1]; memset(dp,
0, sizeof(dp)); for (int i = 1; i <= m; i++) for (int j =
1; j <= n; j++) { if (t1[i-1] == t2[j-1]) dp[i][j] =
dp[i-1][j-1] + 1; else dp[i][j] = dp[i-1][j] > dp[i][j-1]
? dp[i-1][j] : dp[i][j-1]; } return dp[m][n]; }
```

↖ Diagonal = Match ↔ Top/Left = Skip

LCS — Step-by-Step Table Construction

S1 (ROWS) "GAC" S2 (COLS) "AGCAT"

	" "	A	G	C	A	T
" "	0	0	0	0	0	0
G	0	0	0	0	0	0
A	0	0	1	1	**1**	1
C	0	1	1	2	2	2

CASE 1: MISMATCH ✕

Cell: S1[0]='G', S2[1]='A'
'G' != 'A'
dp[1][2] = max(dp[0][2], dp[1][1])
dp[1][2] = max(0, 0) = **0**

CASE 2: MATCH ✓

Cell: S1[1]='A', S2[0]='A'
'A' == 'A'
dp[2][1] = 1 + dp[1][0]
dp[2][1] = 1 + 0 = **1**

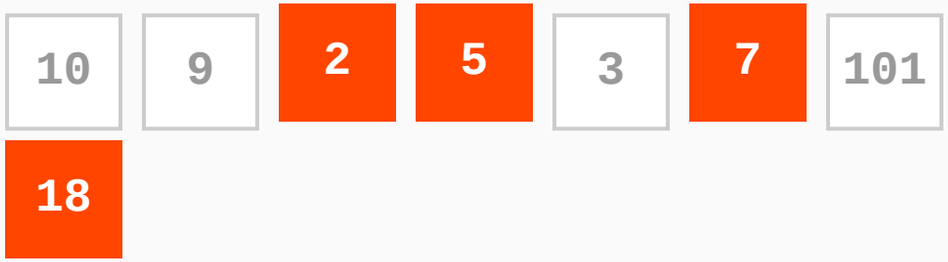
**Table shows partial progress. Final answer is dp[3][5] = 2 ("AC").*

Classic Problem 5 — Longest Increasing Subsequence (LIS)

Problem: Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

- ✓ **Subsequence:** Elements can be skipped, but relative order must be maintained.
- ✓ **Strictly Increasing:** Each element must be strictly greater than the previous one (no duplicates allowed in the chain).

EXAMPLE



LIS: [2, 5, 7, 18]
Length: 4

(Note: [2, 3, 7, 101] is also valid)

WHY NOT JUST SORT?

If we sort the array, we lose the original relative order of elements. The problem requires the subsequence to exist in the **original** array.

LIS — DP Solution

Applying the Recipe

STATE

`dp[i]` = length of LIS **ending at index i**.

RECURRENCE

For all **`j < i`** where **`nums[j] < nums[i]`**:

`dp[i] = max(dp[i], 1 + dp[j])`

BASE CASE

`dp[i] = 1` for all i (every element is an LIS of length 1)

FINAL ANSWER

`max(dp)` — the LIS could end at any index

```
int lengthOfLIS(int nums[], int n) { if (n == 0) return 0; int
dp[n]; for (int i = 0; i < n; i++) dp[i] = 1; for (int i = 1; i
< n; i++) for (int j = 0; j < i; j++) if (nums[j] < nums[i] &&
dp[j]+1 > dp[i]) dp[i] = dp[j] + 1; int ans = 0; for (int i =
0; i < n; i++) if (dp[i] > ans) ans = dp[i]; return ans; }
```

Complexity: $O(n^2)$ Time, O(n) Space.

*There is a more advanced O(n log n) solution using Binary Search (Patience Sorting), but this DP approach is the foundational one.

Classic Problem 6 — Edit Distance

Problem: Find the minimum number of operations required to convert `word1` to `word2`.

ALLOWED OPERATIONS

- Insert a character
- Delete a character
- Replace a character

EXAMPLE: HORSE → ROS

HORSE ↓

RORSE — REPLACE 'H' WITH 'R'

ROSE — DELETE 'R'

ROS — DELETE 'E'

Total Operations: 3

Applications: Spell checking (autocorrect), DNA sequence alignment, Natural Language Processing.

Edit Distance — Recursive Formulation

THE LOGIC

Compare the last characters of strings A and B:

1. **Match:** No cost. Recurse on `(i-1, j-1)`.
2. **Mismatch:** Take min of 3 operations (+1 cost):

- **Insert:** Recurse on `(i, j-1)`
- **Delete:** Recurse on `(i-1, j)`
- **Replace:** Recurse on `(i-1, j-1)`

Recurrence Relation:

```
# If match: ED(i-1, j-1)
# If mismatch:
1 + min(
  ED(i, j-1),   # Insert
  ED(i-1, j),   # Delete
  ED(i-1, j-1)  # Replace
)
```

```
int min3(int a, int b, int c) { if (a < b) return a < c ? a : c; return b < c
? b : c; } int minDistance(char* w1, char* w2, int i, int j) { if (i == 0)
return j; // Insert remaining if (j == 0) return i; // Delete remaining if
(w1[i-1] == w2[j-1]) return minDistance(w1, w2, i-1, j-1); return 1 + min3(
minDistance(w1, w2, i, j-1), // Insert minDistance(w1, w2, i-1, j), // Delete
minDistance(w1, w2, i-1, j-1) // Replace ); }
```

$O(3^m)$ Exponential time complexity.
Worse than LCS — 3 branches at each step.

Edit Distance — DP Solution

Applying the Recipe

STATE

dp[i][j] = min operations to convert **word1[0..i]** to **word2[0..j]**.

RECURRENCE

If match: **dp[i-1][j-1]**

Else: **1 + min(**

dp[i][j-1] (Insert),

dp[i-1][j] (Delete),

dp[i-1][j-1] (Replace)

)

BASE CASE

dp[i][0] = i (Delete all)

dp[0][j] = j (Insert all)

```
int minDistance(char* w1, char* w2) { int m = strlen(w1), n =
strlen(w2); int dp[m+1][n+1]; for (int i = 0; i <= m; i++)
dp[i][0] = i; for (int j = 0; j <= n; j++) dp[0][j] = j; for
(int i = 1; i <= m; i++) for (int j = 1; j <= n; j++) { if
(w1[i-1] == w2[j-1]) dp[i][j] = dp[i-1][j-1]; else dp[i][j] = 1
+ min3( dp[i][j-1], dp[i-1][j], dp[i-1][j-1]); } return dp[m]
[n]; }
```

↖ Replace / Match

← Insert

↑ Delete

Edit Distance — Table Walkthrough

WORD1 (ROWS) "HORSE" **WORD2 (COLS)** "ROS"

	" "	R	O	S
" "	0	1	2	3
H	1	1	2	3
O	2	2	**1**	2
R	3	2	2	2
S	4	3	3	2
E	5	4	4	**3**

MISMATCH EXAMPLE

Cell: 'H' vs 'R'

1 + min(0, 1, 1) = **1**

(Replace 'H' with 'R')

MATCH EXAMPLE

Cell: 'O' vs 'O'

Take diagonal value directly.

dp[2][2] = dp[1][1] = **1**

FINAL ANSWER

dp[5][3] = **3**

(HORSE → ROS)

Classic Problem 7 — Grid Paths with Obstacles

Problem: A robot starts at top-left (0,0) and wants to reach bottom-right. It can only move **Down** or **Right**. Some cells are obstacles. How many unique paths exist?

RECURRENCE LOGIC

If grid[i][j] is obstacle:

```
dp[i][j] = 0
```

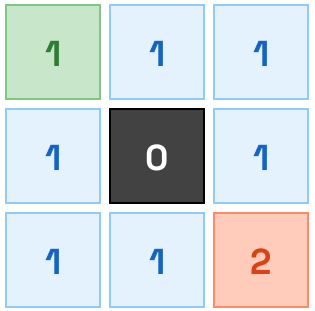
Else:

```
dp[i][j] = dp[i-1][j] + dp[i][j-1]
```

(Sum of paths from top and left)

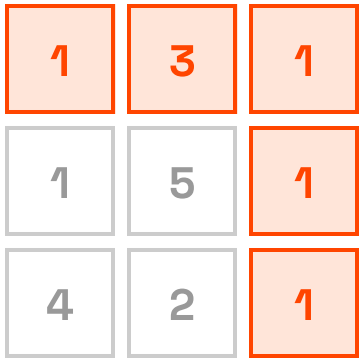
```
int uniquePathsWithObstacles(int grid[][100], int m, int n) {
int dp[m][n]; memset(dp, 0, sizeof(dp)); if (grid[0][0] == 0)
dp[0][0] = 1; for (int i = 0; i < m; i++) for (int j = 0; j <
n; j++) { if (grid[i][j] == 1) dp[i][j] = 0; else { if (i > 0)
dp[i][j] += dp[i-1][j]; if (j > 0) dp[i][j] += dp[i][j-1]; } }
return dp[m-1][n-1]; }
```

Example DP Table (1 = Path Count)



Classic Problem 8 — Minimum Path Sum

Problem: Find a path from top-left to bottom-right which minimizes the sum of all numbers along its path. You can only move either **down** or **right**.



Path: 1 → 3 → 1 → 1 → 1 = 7

RECURRENCE RELATION

```
dp[i][j] = grid[i][j] + min(
  dp[i-1][j], # from top
  dp[i][j-1] # from left
)
```

```
int minPathSum(int grid[][100], int m, int n) { for (int i = 0; i < m; i++) for (int j = 0; j < n; j++) { if (i == 0 && j == 0) continue; else if (i == 0) grid[i][j] += grid[i][j-1]; else if (j == 0) grid[i][j] += grid[i-1][j]; else { int mn = grid[i-1][j] < grid[i][j-1] ? grid[i-1][j] : grid[i][j-1]; grid[i][j] += mn; } } return grid[m-1][n-1]; }
```

Classic Problem 9 — House Robber

Problem: You are a robber planning to rob houses along a street. Each house has a certain amount of money stashed.

THE CONSTRAINT

Adjacent houses have connected security systems. You **cannot rob two adjacent houses** on the same night.

The Choice (Recurrence)

```
dp[i] = max(
  dp[i-1], // Skip house i
  nums[i] + dp[i-2] // Rob house i
)
```

```
int rob(int nums[], int n) { if (n == 0) return 0; if
(n == 1) return nums[0]; int dp[n]; dp[0] = nums[0];
dp[1] = nums[0] > nums[1] ? nums[0] : nums[1]; for
(int i = 2; i < n; i++) { int take = nums[i] + dp[i-
2]; dp[i] = take > dp[i-1] ? take : dp[i-1]; } return
dp[n-1]; }
```

Example: [2, 7, 9, 3, 1]

Total: 2 + 9 + 1 = **12**

Classic Problem 10 — Maximum Subarray

Problem: Find the contiguous subarray (containing at least one number) which has the largest sum.

THE CORE DECISION

At each element `nums[i]`, we ask:
"Should I start a new subarray here, or extend the previous one?"

```
current_sum = max(nums[i], current_sum + nums[i])
max_sum = max(max_sum, current_sum)
```

EXAMPLE

Max Sum: 6 (from [4, -1, 2, 1])

```
int maxSubArray(int nums[], int n) { int max_sum = nums[0]; int current = nums[0]; for (int i = 1; i < n; i++) { // Extend or start fresh? current = nums[i] > current + nums[i] ? nums[i] : current + nums[i]; if (current > max_sum) max_sum = current; } return max_sum; }
```

Time: O(n) **Space:** O(1)

Advanced — Matrix Chain Multiplication

THE PROBLEM

Matrix multiplication is **associative**:
(AB)C = A(BC).

However, the **cost** (number of scalar multiplications) depends heavily on the order of parenthesization.

THE COST MODEL

Multiplying a matrix **p×q** by **q×r** requires:

```
Cost = p * q * r operations
```

EXAMPLE: A, B, C

A	B	C
10×30	30×5	5×60

Option 1: (AB)C

`(10*30*5) + (10*5*60) = 1500 + 3000`

4,500 ops ✓

Option 2: A(BC)

`(30*5*60) + (10*30*60) = 9000 + 18000`

27,000 ops ✗

6x Slower! Order matters.

Matrix Chain Multiplication — DP Solution

Applying the Recipe

STATE

dp[i][j] = min scalar multiplications needed for chain $A_i...A_j$.

RECURRENCE

Try every split point **k** between **i** and **j**:

```
dp[i][j] = min(
 dp[i][k] + dp[k+1][j]
 + p[i-1]*p[k]*p[j]
)
```

BASE CASE

dp[i][i] = 0

(Single matrix requires 0 ops)

```
int matrixChainOrder(int p[], int size) { int n = size - 1; int
dp[n+1][n+1]; memset(dp, 0, sizeof(dp)); for (int l = 2; l <= n; l++)
// chain length for (int i = 1; i <= n-l+1; i++) { int j = i + l - 1;
dp[i][j] = INT_MAX; for (int k = i; k < j; k++) { int cost = dp[i][k]
+ dp[k+1][j] + p[i-1]*p[k]*p[j]; if (cost < dp[i][j]) dp[i][j] =
cost; } } return dp[1][n]; }
```

$O(n^3)$ Interval DP Pattern.
Three nested loops: Length, Start Index, Split Point.

Recognizing DP Problems

1. OPTIMIZATION

- → Maximize Value
  Profit, Loot, Subsequence Length
- → Minimize Cost
  Path Sum, Edit Distance, Coins
- → "Longest" / "Shortest"

2. COUNTING

- → Number of Ways
  Grid Paths, Climbing Stairs
- → Count Distinct
  Arrangements, Partitions

3. FEASIBILITY

- → Is it possible?
  Subset Sum, Interleaving String
- → Can we reach?
  Jump Game (often Greedy, but DP too)

THE CONSTRAINT CHECK (INPUT SIZE N)

N ~ 10-20

$O(2^N)$ or $O(N!)$
Recursion / Backtracking

N ~ 100-1000

$O(N^2)$ or $O(N^3)$
Dynamic Programming

N ~ 10^5+

$O(N)$ or $O(N \log N)$
Greedy / Divide & Conquer

Common DP Patterns — A Reference Table

PATTERN	TYPICAL STATE	COMPLEXITY	EXAMPLES
Linear (1D)	`dp[i]` depends on `dp[i-k]`	**`O(N)`**	Climbing Stairs, House Robber, Fibonacci
Grid (2D)	`dp[i][j]` depends on `dp[i-1][j]`, `dp[i][j-1]`	**`O(N*M)`**	Unique Paths, Min Path Sum
Knapsack	`dp[i][w]` (item i, weight w)	**`O(N*W)`**	0/1 Knapsack, Subset Sum, Partition Equal Subset
String Alignment	`dp[i][j]` (suffix/prefix i, j)	**`O(N*M)`**	LCS, Edit Distance, Interleaving String
Unbounded Knapsack	`dp[w]` (capacity w)	**`O(N*W)` or `O(W)`**	Coin Change, Rod Cutting

Bonus Problem — Longest Palindromic Subsequence

Pattern: Range DP

> **Problem:** Given a string `s`, find the length of the longest palindromic subsequence.

RECURRENCE RELATION

State: `dp[i][j] = LPS length in s[i...j]`

If s[i] == s[j]:

```
2 + dp[i+1][j-1]
```

Else:

```
max(dp[i+1][j], dp[i][j-1])
```

```c
int longestPalinSubseq(char* s) { int n = strlen(s); int
dp[n][n]; memset(dp, 0, sizeof(dp)); for (int i = n-1; i
>= 0; i--) { dp[i][i] = 1; for (int j = i+1; j < n; j++)
{ if (s[i] == s[j]) dp[i][j] = dp[i+1][j-1] + 2; else
dp[i][j] = dp[i+1][j] > dp[i][j-1] ? dp[i+1][j] : dp[i]
[j-1]; } } return dp[0][n-1]; }
```

Example: "bbbab"

LPS: "bbbb" (Length 4)

Bonus Problem — Subset Sum

Pattern: 0/1 Knapsack (Feasibility)

> **Problem:** Given a set of non-negative integers and a value sum, determine if there is a subset of the given set with sum equal to given sum.

RECURRENCE RELATION

State: dp[i][j] = True if sum 'j' is possible using first 'i' items.

Logic:

```
dp[i][j] =
  dp[i-1][j] // Exclude
  OR
  dp[i-1][j - nums[i]] // Include
```

```
#include <stdbool.h> bool isSubsetSum(int nums[], int n,
int target) { // dp[j] = true if sum j is possible bool
dp[target + 1]; memset(dp, false, sizeof(dp)); dp[0] =
true; // Sum 0 always possible for (int i = 0; i < n;
i++) for (int j = target; j >= nums[i]; j--) if (dp[j -
nums[i]]) dp[j] = true; return dp[target]; }
```

Example: Target = 9

3	34	**4**	12	**5**	2

Result: **True** (4 + 5 = 9)

Common Mistakes & Pitfalls

THE GREEDY TRAP

Assuming a locally optimal choice leads to a global optimum. DP is needed when future consequences matter.

```
Example: Coin Change with coins [1, 3, 4].
Target 6: Greedy takes 4+1+1 (3 coins).
Optimal is 3+3 (2 coins).
```

INCOMPLETE STATE

Failing to include all changing parameters in the DP state key. If it affects the future, it must be in the state.

```
Example: "Buy/Sell Stock" problems.
Wrong: dp[day]
Right: dp[day][has_stock]
```

OFF-BY-ONE ERRORS

Incorrect table sizing or loop boundaries. DP tables often need size N+1 to handle base cases (like empty string/array).

```
Example: Edit Distance for strings of len M, N.
Table needs size (M+1) x (N+1).
```

WRONG DIRECTION

Iterating in the wrong order. You cannot calculate a state if its dependencies haven't been computed yet.

```
Example: 0/1 Knapsack with 1D array.
Must iterate backwards to avoid reusing items.
Iterating forwards turns it into Unbounded
Knapsack.
```

The DP Mastery Roadmap

1

RECURSION & MEMOIZATION

Understand the call stack, overlapping subproblems, and caching results.

Fibonacci, Climbing Stairs

2

1D DYNAMIC PROGRAMMING

Linear dependencies. Solving for i based on i-1, i-2.

House Robber, Maximum Subarray

3

2D GRIDS & MATRICES

Spatial dependencies. Moving through a grid.

Unique Paths, Min Path Sum

4

CLASSIC MULTI-VARIABLE PATTERNS

Two strings, knapsack weights, complex state transitions.

LCS, Edit Distance, 0/1 Knapsack, LIS

5

ADVANCED DP

Intervals, Bitmasks, Trees, and Probability.

Matrix Chain Mult, TSP (Bitmask), Tree Diameter

Summary — Key Takeaways

THE PHILOSOPHY

Smart Recursion

DP is essentially recursion with caching. We trade space for time to avoid re-calculating the same subproblems.

Two Pillars

Overlapping Subproblems: The same small problems appear repeatedly.
Optimal Substructure: Optimal solution can be built from optimal sub-solutions.

The Impact

Reduces complexity from Exponential $O(2^N)$ to Polynomial $O(N^k)$.

THE METHODOLOGY

The 4-Step Recipe

1. Define the **State** (dp[i]).
2. Find the **Recurrence** (Transitions).
3. Identify **Base Cases**.
4. Determine **Order** of computation.

Two Approaches

Memoization (Top-Down): Easy to write, good for sparse states.
Tabulation (Bottom-Up): Iterative, no recursion overhead, enables space optimization.

> **"Those who cannot remember the past are condemned to repeat it."**

Practice Problems & Resources

LEVEL 1: WARM-UP (1D DP)

- Climbing Stairs
- House Robber
- Min Cost Climbing Stairs
- Maximum Subarray

LEVEL 2: CORE PATTERNS (2D / GRID)

- Unique Paths I & II
- Coin Change
- Longest Increasing Subsequence
- Minimum Path Sum
- Longest Common Subsequence
- Edit Distance

LEVEL 3: CHALLENGE

- Burst Balloons
- Palindrome Partitioning II
- Regular Expression Matching
- Best Time to Buy/Sell Stock IV

Books

CLRS (Intro to Algorithms)
Chapter 15: Dynamic Programming

Grokking Algorithms
Chapter 9: Dynamic Programming

Platforms

LeetCode (Tag: Dynamic Programming)

HackerRank (Algorithms / DP)

VisualAlgo.net

> **Pro Tip:** Don't just memorize solutions. Draw the recursion tree and the DP table for every problem you solve.

Thank You

Dynamic Programming: From Recursion to Optimization

QUESTIONS?